

Теорія інформації та кодування

Лекція 9. Системи стиснення інформації. Словникові методи стиснення

Основою словникових алгоритмів є алгоритм LZ, названий за ініціалами своїх розробників: Яків Зів (Jacob Ziv) та Абрахам Лемпел (Abraham Lempel).

Словникові алгоритми Лемпела–Зіва поділяють на дві сім'ї:

1. До першої сім'ї належать алгоритми, у яких весь словник створюється адаптивно (тобто в процесі стиснення) і не зберігається, а в неявному вигляді міститься у закодованих даних. Зберігаються лише неповторювані символи та вказівники на повторювані фрази, що трапляються у повідомленні. Ми розглянемо два алгоритми: базовий алгоритм LZ77 та один з найдосконаліших представників цієї сім'ї – алгоритм LZSS (Й. Сторер (Storer) і Т. Шиманський (Szimanski)).

2. До другої сім'ї належать алгоритми, які початковий словник джерела доповнюють новими фразами, що є повторюваними у повідомленні комбінаціями символів початкового словника та зберігають цей доповнений словник поряд з індексом фрази або окремого символа, що є кодом стисненого повідомлення. Ми розглянемо два алгоритми: базовий алгоритм LZ78 та один з найбільш досконалих представників цієї сім'ї – алгоритм LZW (Т. Уелч (Welch)).

Алгоритм LZ77.

Алгоритми цієї сім'ї ще називають алгоритмами з ковзним словником або ковзним вікном.

У процесі кодування положення словника (вікна) відносно початку повідомлення постійно змінюється, словник наче ковзає по вхідному повідомленню. Щоб досягти стиснення, він намагається замінити черговий фрагмент повідомлення (зазвичай, найдовшу фразу) вказівником на таку ж фразу зі словника.

Ковзне вікно має довжину N , і розбите на дві нерівномірні частини:

- перша, більша за розміром, містить уже прочитану частину повідомлення, її називають словником;
- друга, набагато менша, називають буфером або випереджувальним буфером, який містить частину ще не закодованого повідомлення.

Зазвичай, розмір словника M становить декілька кілобайтів, а розмір буфера F набагато менший – до 100 байтів.

Під час кодування вмісту буфера серед попередніх $N - F$ символів, тобто у словнику, відбувається пошук найдовшої фрази (послідовність символів), що збігається з фразою, яка є на початку буфера.

Знайдений найбільший збіг кодує тріада $\langle i, j, s \rangle$, де

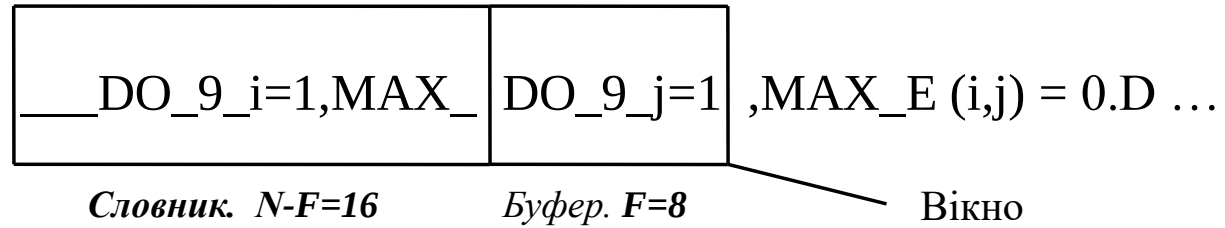
i — зсув у словнику фрази, що збігається з фразою на початку буфера;

j — довжина фрази, що збігається;

s — перший символ, який є після фрази, що збігається, в буфері.

Далі алгоритм зсовує увесь вміст вікна на $j + 1$ символів і водночас зчитує стільки ж символів вхідного потоку у буфер.

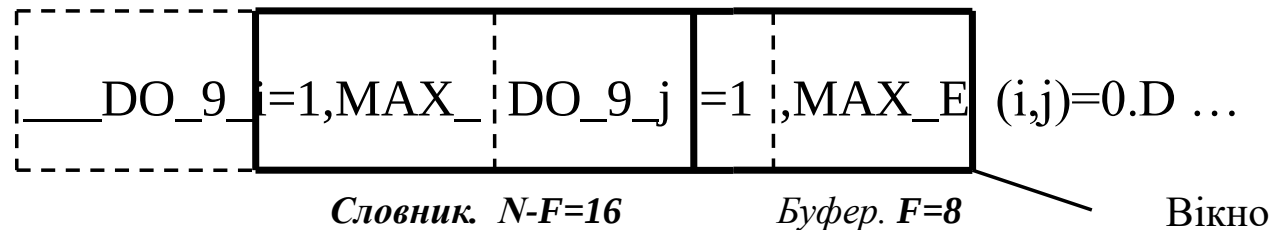
Розглянемо роботу (один крок) алгоритму LZ77 на прикладі такого фрагмента повідомлення:



Найдовша фраза (послідовність символів), яка співпадає з фразою на початку буфера, є “DO_9_”.

Вихідний код матиме вигляд $\langle 3, 5, j \rangle$.

Після цього алгоритм зсуває весь вміст вікна на довжину фрази +1 символ ($5 + 1 = 6$ символів) уліво й одночасно зчитує відповідну кількість (6) чергових символів з повідомлення у буфер.



Якщо на деякому кроці збігу (деякої фрази чи бодай одного символу, що міститься на початку буфера) не виявлено, то алгоритм видає код $\langle 0, 0, \text{перший символ у буфері} \rangle$ та продовжує роботу.

Обсяг пам'яті, що її потребує алгоритм-кодер або декодер, визначений розміром вікна N .

Довжину коду обчислюють так:

довжина фрази, що збіглася з вмістом словника, не може бути більшою від розміру буфера F мінус 1, а зсув цієї фрази у словнику не може бути більшим від розміру словника мінус 1;

довжина двійкового коду зсуву i буде $\lceil \log_2 (N - F) \rceil$

довжина коду довжини фрази j буде $\lceil \log_2 F \rceil$

символ s кодується 8 бітами за таблицею ASCII+.

Приклад. Стиснемо за алгоритмом LZ77 повідомлення “А-БА-БА-ГА-ЛА-МА-ГА”; розмір словника – 8 байтів, буфера – 5 байтів.

Словник (8 байт)								Буфер (5 байт)					Код
0	1	2	3	4	5	6	7	1	2	3	4	5	
								А	-	Б	А	-	<0,0,'А'>
							А	-	Б	А	-	Б	<0,0,'-'>
						А	-	Б	А	-	Б	А	<0,0,'Б'>
					А	-	Б	А	-	Б	А	-	<5,3,'А'>
	А	-	Б	А	-	Б	А	-	Г	А	-	Л	<5,1,'Г'>
-	Б	А	-	Б	А	-	Г	А	-	Л	А	-	<5,2,'Л'>
-	Б	А	-	Г	А	-	Л	А	-	М	А	-	<5,2,'М'>
-	Г	А	-	Л	А	-	М	А	-	Г	А		<5,2,'Г'>
-	Л	А	-	М	А	-	Г	А					<0,0,'А'>

В останньому рядку таблиці буква “А” взята не зі словника, оскільки вона остання.

Довжина отриманого коду $L_{LZ77} = 9 \cdot (3 + 3 + 8) = 126$ біт, $L_{ASCII+} = 19 \cdot 8 = 152$ біти.

Приклад. Стиснемо за алгоритмом LZ77 повідомлення “АБРАКАДАБРА”; розмір словника – 8 байтів, буфера – 5 байтів.

Словник (8 байт)								Буфер (5 байт)					Код
0	1	2	3	4	5	6	7	1	2	3	4	5	
								А	Б	Р	А	К	<0,0,'А'>
							А	Б	Р	А	К	А	<0,0,'Б'>
						А	Б	Р	А	К	А	Д	<0,0,'Р'>
					А	Б	Р	А	К	А	Д	А	<5,1,'К'>
			А	Б	Р	А	К	А	Д	А	Б	Р	<3,1,'Д'>
-	А	Б	Р	А	К	А	Д	А	Б	Р	А		<1,2,'Р'>
Р	А	К	А	Д	А	Б	Р	А					<0,0,'А'>

Довжина отриманого коду $L_{LZ77} = 7 \cdot (3 + 3 + 8) = 98$ біт, $L_{ASCII+} = 11 \cdot 8 = 88$ біт.

Під час декодування виконується той самий порядок роботи з вікном, що й під час кодування, однак, на відміну від пошуку підрядків, що співпадають, їх, навпаки, копіює декодер з вікна згідно з черговою тріадою коду.

Приклад. Декодуємо повідомлення, заповане за алгоритмом LZ77, довжина словника – 8 байтів. Код стисненого повідомлення:

$$\langle 0,0,'A' \rangle \langle 0,0,'-' \rangle \langle 0,0,'Б' \rangle \langle 5,3,'A' \rangle \langle 5,1,'Г' \rangle \langle 5,2,'Л' \rangle \langle 5,2,'М' \rangle \langle 5,2,'Г' \rangle \langle 0,0,'A' \rangle.$$
[illegible]

Проблеми:

1. Швидкодія. Значно залежить від того, як відбуватиметься пошук однакових фраз у словнику. Якщо шукати збіг повним перебиранням можливих варіантів, то очевидно, що стискання буде дуже повільним.
2. Стисканням інформації. З'являється тоді, коли кодер не може знайти тотожної фрази у словнику і видає стандартну тріаду, кодуючи один символ, наприклад, якщо словник має довжину $4K$, а буфер – 16 байтів, то код, який видає кодер $\langle 0, 0, \text{символ} \rangle$, займатиме 3 байти

Алгоритм LZSS.

Алгоритм LZSS відрізняється від алгоритму LZ77 тим, як підтримується ковзне вікно, та кодами кодера.

Код алгоритму починається однобітовим префіксом, який відділяє код фрази, що збігається, від незакодованого символу.

Код складається з пари $\langle i, j \rangle$ — зсуву i у словнику фрази, що збігається з фразою на початку буфера, і довжини j цієї фрази, як і для алгоритму LZ77.

Вікно зсувається рівно на довжину знайденого підрядка або на 1, якщо входження підрядка буфера у словнику не знайдено.

Довжина фрази у алгоритмі LZSS завжди більше 0 і не може перевищувати розмір буфера F , тому довжина двійкового коду довжини фрази, що збігається, j буде $\lceil \log_2(F) \rceil$, а довжина коду зсуву i — $\lceil \log_2(N - F) \rceil$.

Приклад. Стиснемо за алгоритмом LZSS повідомлення “А-БА-БА-ГА-ЛА-МА-ГА”; розмір словника – 8 байтів, буфера – 5 байтів.

Словник (8 байт)								Буфер (5 байт)					Код
0	1	2	3	4	5	6	7	1	2	3	4	5	
								А	-	Б	А	-	0’А’
							А	-	Б	А	-	Б	0’-’
						А	-	Б	А	-	Б	А	0’Б’
					А	-	Б	А	-	Б	А	-	1<5,3>
		А	-	Б	А	-	Б	А	-	Г	А	-	1<5,2>
А	-	Б	А	-	Б	А	-	Г	А	-	Л	А	0’Г’
-	Б	А	-	Б	А	-	Г	А	-	Л	А	-	1<5,2’>
А	-	Б	А	-	Г	А	-	Л	А	-	М	А	0’Л’
-	Б	А	-	Г	А	-	Л	А	-	М	А	-	1<5,2>
А	-	Г	А	-	Л	А	-	М	А	-	Г	А	0’М’
-	Г	А	-	Л	А	-	М	А	-	Г	А		1<5,2>
А	-	Л	А	-	М	А	-	Г	А				0’Г’
-	Л	А	-	М	А	-	Г	А					0’А’

Довжина отриманого коду $L_{LZSS} = 8 \cdot (1+ 8) + 5 \cdot (1+ 3 + 3) = 107$ біт, $L_{ASCII+} = 152$ біт, $L_{LZ77} = 126$ біт

Приклад. Стиснемо за алгоритмом LZSS повідомлення “АБРАКАДАБРА”; розмір словника – 8 байтів, буфера – 5 байтів.

Словник (8 байт)								Буфер (5 байт)					Код
0	1	2	3	4	5	6	7	1	2	3	4	5	
								А	Б	Р	А	К	0’А’
							А	Б	Р	А	К	А	0’Б’
						А	Б	Р	А	К	А	Д	0’Р’
					А	Б	Р	А	К	А	Д	А	1<5,1>
				А	Б	Р	А	К	А	Д	А	Б	0’К’
			А	Б	Р	А	К	А	Д	А	Б	Р	1<6,1>
		А	Б	Р	А	К	А	Д	А	Б	Р	А	0’Д’
	А	Б	Р	А	К	А	Д	А	Б	Р	А		1<1,3 >
Р	А	К	А	Д	А	Б	Р	А					0’А’

Довжина отриманого коду $L_{LZSS} = 6 \cdot (1+ 8) + 3 \cdot (1+ 3 + 3) = 75$ біт, $L_{ASCII+} = 88$ біт, $L_{LZ77} = 98$ біт

Алгоритм LZSS є дуже асиметричним за часом. Процедура стискання достатньо складна, а робота декодера алгоритму LZSS є тривіально простою і може виконуватися зі швидкістю, яка наближається до швидкості процедури звичайного копіювання інформації.

Декодування:

1. Декодер читає один біт стисненої інформації (префікс коду) і визначає, чи це символ, чи пара <зсув, довжина> (0 – це символ, 1 – пара).
2. Якщо це символ, то наступні 8 бітів видаються як розкодований символ і заносяться у ковзне вікно. Якщо це пара, то відповідна кількість символів словника заносяться у ковзне вікно та видається у розкодованому вигляді.

Приклад. Декодуємо повідомлення, закодоване за алгоритмом LZSS, довжина словника – 8 байтів. Код стисненого повідомлення:

0’А’0’Б’0’Р’1<5,1>0’К’1<6,1>0’Д’1<1,3 >0’А’.

Вхідний код	Вихідний символ	Словник (8 байт)							
		0	1	2	3	4	5	6	7
0’А’	А								А
0’Б’	Б							А	Б
0’Р’	Р						А	Б	Р
1<5,1>	А					А	Б	Р	А
0’К’	К				А	Б	Р	А	К
1<6,1>	А			А	Б	Р	А	К	А
0’Д’	Д		А	Б	Р	А	К	А	Д
1<1,3 >	АБР	Р	А	К	А	Д	А	Б	Р
0’А’	А	А	К	А	Д	А	Б	Р	А

Недоліки:

1. Розглянуті алгоритми LZ77, LZSS як словник використовують лише невеликий фрагмент повідомлення. Це не дає змоги кодувати повторювані фрази, відстань між якими у повідомленні є більшою, ніж розмір словника.
2. Розмір фрази, яку можна закодувати обмежений розміром буфера.

Якщо ми збільшимо розмір словника з 4 до 64К, а розмір буфера з 16 байт до 1К, то для кодування зсуву у словнику потрібно буде 16 біт замість 12, а для кодування довжини фрази – 10 біт замість 4.

Також зросте час роботи алгоритму кодера. Збільшення розмірів словника з 4 до 64К в алгоритмі LZ77 приведе до зростання в 16 разів часу пошуку найдовшого збігу, а зі збільшенням розміру буфера з 16 байт до 1К ця операція виконуватиметься в 64 рази довше для обох алгоритмів.

Алгоритм LZ78.

Цей алгоритм не використовує вікно, а зберігає словник із фраз повідомлення, що вже проглянуте.

Кодер і декодер починають роботу з майже порожнього словника, який містить тільки порожній рядок у позиції нуль.

У міру надходження та кодування символів нові фрази (окремі символи) додаються в позиції 1, 2 і т. д.

Коли кодер зчитує черговий символ повідомлення, символ додається до накопичувального рядка. Доти, доки накопичувальний рядок відповідає який-небудь фразі словника, процес продовжується.

Як тільки накопичувальний рядок перестає відповідати якійсь фразі словника, кодер LZ78 генерує код з індексу фрази у словнику, яка до останнього зчитаного символа збіглася з накопичувальним рядком, і символа, що порушив збіг.

Нова фраза (накопичувальний рядок) додається у словник.

Для визначення розміру кодів ключовим є розмір словника, оскільки кожний код за алгоритмом LZ78 містить номер фрази у словнику.

Отже коди алгоритму мають сталу довжину, яка дорівнює округленому до більшого цілого

$$\lceil \log_2([\textit{розмір словника}]) \rceil + 8$$

Приклад. Стиснемо за алгоритмом LZ78 повідомлення “А-БА-БА-ГА-ЛА-МА-ГА”; використовуючи словник довжиною 16 фраз.

Вхідна фраза (словник)	Код	Індекс фрази
' '		0
А	<0,'А'>	1
-	<0,' - '>	2
Б	<0,'Б'>	3
А-	<1,' - '>	4
БА	<3,'А'>	5
-Г	<2,'Г'>	6
А-Л	<4,'Л'>	7
А-М	<4,'М'>	8
А-Г	<4,'Г'>	9
А	<0,'А'>	10

Довжина отриманого коду $L_{LZ78} = 10 \cdot (4 + 8) = 120$ біт, $L_{ASCII+} = 152$ біти.

Приклад. Стиснемо за алгоритмом LZ78 повідомлення “АБРАКАДАБРА”, використовуючи словник довжиною 16 фраз.

Вхідна фраза (словник)	Код	Індекс фрази
' '		0
А	<0,'А'>	1
Б	<0,'Б'>	2
Р	<0,'Р'>	3
АК	<1,'К'>	4
АД	<1,'Д'>	5
АБ	<1,'Б'>	6
РА	<3,'А'>	7

Довжина отриманого коду $L_{LZ78} = 7 \cdot (4 + 8) = 84$ біти, $L_{ASCII+} = 88$ біт.

Приклад. Розкодуємо повідомлення

<0,'А'><0,'-'><0,'Б'><1,'-'><3,'А'><2,'Г'> <4,'Л'><4,'М'><4,'Г'><0,'А'> ,

закодоване за алгоритмом LZ78. Довжина словника – 16 фраз.

Вхідна код	Вихід (словник)	Індекс фрази
' '		0
<0,'А'>	А	1
<0,' - '>	-	2
<0,'Б'>	Б	3
<1,' - '>	А-	4
<3,'А'>	БА	5
<2,'Г'>	-Г	6
<4,'Л'>	А-Л	7
<4,'М'>	А-М	8
<4,'Г'>	А-Г	9
<0,'А'>	А	10

Алгоритм LZW.

Відрізняється від LZ78 приблизно тим, чим LZSS відрізняється від LZ77, а саме: довжина коду зменшується на одне поле, тобто кодер LZW ніколи не видає самі символи повідомлення, яке стискається, а видає лише вказівники на місце символу або фрази у словнику.

Робота алгоритму LZW починається ініціалізацією словника, наприклад, розміром 4К всіма символами вхідного алфавіту (таблиця ASCII+) за адресами від 0 до 255. А адреси від 256 до 4095 заповнюватимуться фразами у процесі стискання.

Оскільки словник уже частково заповнений, то перші зчитані символи повідомлення завжди будуть знайдені у словнику. Тому коди складаються тільки з вказівника на символ або фразу у словнику (адреса символу або фрази у словнику) і мають фіксовану довжину, що визначена величиною

$$\lceil \log_2 ([\textit{розмір словника}]) \rceil$$

У процесі кодування алгоритм LZW додає символи, які зчитуються, до накопичувального рядка w і після кожного нового доданого символа, кодер шукає w у словнику. Якщо рядок знайдено, то процес подовження w продовжується. Як тільки додавання нового символа S приводить до того, що рядка wS у словнику немає, тоді кодер:

- 1) записує у вихідний файл вказівник, під яким рядок w міститься у словнику;
- 2) зберігає накопичувальний рядок wS у словнику на наступній допустимій позиції (присвоює наступний вільний вказівник);
- 3) присвоює (ініціалізує) накопичувальному рядку w нове значення – S .

Формально:

Крок 1. Ініціалізація словника всіма можливими односимвольними фразами (звичайно це 256 символів ASCII+). Ініціалізація накопичувального рядка w першим символом повідомлення.

Крок 2. Зчитування наступного символу S повідомлення.

Крок 3.

ЯКЩО S це символ КІНЕЦЬ_ПОВІДОМЛЕННЯ

Видати код для w

ПЕРЕЙТИ на КІНЕЦЬ.

Кінець ЯКЩО

ЯКЩО фраза wS вже є у словнику

Привласнити вхідній фразі w значення wS

ПЕРЕЙТИ до кроку 2

ІНАКШЕ

Видати код w

Додати wS у словник

Привласнити вхідній фразі wS значення символу S

ПЕРЕЙТИ до кроку 2.

Кінець ЯКЩО

КІНЕЦЬ

Приклад. Стиснемо за алгоритмом LZW повідомлення “А-БА-БА-ГА-ЛА-МА-ГА”; використовуючи словник довжиною 500 фраз.

Накопичувальна фраза wS (словник)	Вказівник фрази	Код фрази w
ASCII+	0–255	
А-	256	192 (А)
-Б	257	045 (-)
БА	258	193 (Б)
А-Б	259	256
БА-	260	258
-Г	261	045 (-)
ГА	262	195 (Г)
А-Л	263	256
ЛА	264	203 (Л)
А-М	265	256
МА	266	204 (М)
А-Г	267	256
ГА		262

Отримаємо:

<192><045><193><256> <258><045><195><256><203><256> <204><256><262>

Довжина отриманого коду $L_{LZW} = 13 \cdot \lceil \log_2 500 \rceil = 117$ біт, $L_{LZ78} = 10 \cdot (4 + 8) = 120$ біт, $L_{ASCII+} = 152$ біти.

Приклад . Стиснемо за алгоритмом LZW повідомлення “АБРАКАДАБРА”; використовуючи словник довжиною 500 фраз.

Накопичувальна фраза wS (словник)	Вказівник фрази	Код фрази w
ASCII+	0–255	
АБ	256	192 (А)
БР	257	193 (Б)
РА	258	208 (Р)
АК	259	192 (А)
КА	260	202 (К)
АД	261	192 (А)
ДА	262	196 (Д)
АБР	263	256
РА		258

Отримаємо:

<192><193><208><192> <202><192><196><256><258>.

Довжина отриманого коду $L_{LZW} = 9 \cdot \lceil \log_2 500 \rceil = 81$ біт, $L_{LZ78} = 10 \cdot (4 + 8) = 84$ біти, $L_{ASCII+} = 88$ біт.

Декодер починає роботу з заповнення словника символами ASCII.

Далі він читає вхідний файл, який складається з вказівників у словнику і використовує кожний вказівник для відновлення нестиснених символів зі словника та записує їх у вихідний файл. У процесі декодування відбувається побудова (доповнення) словника аналогічно як під час стиснення.

Отже:

На першому кроці декодування декодер зчитує перший вказівник і використовує його для відновлення словникового елемента w .

Далі потрібно записати у словник фразу wS однак символ S ще невідомий (це буде перший символ наступної фрази, яка береться зі словника). Для цього на кожному кроці декодування, окрім першого, декодер зчитує з вхідного файлу наступний вказівник, за ним визначає наступний символ (фразу) z зі словника, який записує у вихідний файл. З фрази z виділяє перший символ S формує фразу wS і заносить її у словник на вільне місце (попередньо перевіривши відсутність фрази wS у словнику). Переславши z у w декодер переходить до зчитування наступного вказівника з вхідного файлу і т. д

Приклад. Розпакуємо повідомлення

$\langle 192 \rangle \langle 045 \rangle \langle 193 \rangle \langle 256 \rangle \langle 258 \rangle \langle 045 \rangle \langle 195 \rangle \langle 256 \rangle \langle 203 \rangle \langle 256 \rangle \langle 204 \rangle \langle 256 \rangle \langle 262 \rangle$,

закодоване за алгоритмом LZW. Довжина словника – 500 фраз.

Стиснений файл буде розпакований у такий спосіб.

Спочатку відбувається заповнення словника символами ASCII з 0 по 255 позицію, а далі – поповнення словника та формування вихідного (розпакованого) файлу за допомогою таких дій.

Зчитується перший вказівник – 192. Зі словника береться $w = \text{“A”}$ і виводиться у вихідний файл символ “A”.

Вихідний файл: “A”

Накопичувальний рядок wS потрібно записати у словник, проте S є невідомим. Тому на цьому кроці у словник нічого не заноситься.

Зчитується наступний вказівник – 45. Зі словника береться $z = \text{“-”}$ і виводиться у вихідний файл символ “-”.

Вихідний файл: “А-”

Оскільки z складається з одного символу, то формується фраза $wS = \text{“А-”}$ і вона заноситься у словник під номером 256. Крім того, пересилається z у w , тобто $w = \text{“-”}$ і декодер переходить до зчитування наступного вказівника.

Зчитується наступний вказівник – 193. Зі словника береться $z = \text{“Б”}$ і виводиться у вихідний файл символ “Б”.

Вихідний файл: “А-Б”

Знову ж таки, оскільки z складається з одного символу, то формується фраза $wS = \text{“-Б”}$ і заноситься у словник під номером 257. Пересилається z у w , тобто $w = \text{“Б”}$.

Зчитується наступний вказівник – 256. Зі словника береться фраза $z = \text{“А-”}$ (ця фраза записана у словник декодером на попередніх кроках) і виводиться у вихідний файл.

Вихідний файл: “А-БА-”

Оскільки тепер z складається з декількох (двох) символів, то з z виділяється перший символ $S = \text{“А”}$ і формується фраза $wS = \text{“БА”}$, яка заноситься у словник під номером 258. Пересилається z у w , тобто $w = \text{“А-”}$.

Зчитується наступний вказівник – 258. Зі словника береться фраза $z = \text{“БА”}$ і виводиться у вихідний файл.

Вихідний файл: “А-БА-БА

З фрази z виділяється перший символ $S = \text{“Б”}$ і формується фраза $wS = \text{“А-Б”}$, яка заноситься у словник під номером 259. Пересилається z у w , тобто $w = \text{“БА”}$.

Вхідний код	Вихід	Словник	Індекс фрази
		ASCII	0 – 255
<192>	А		
<045>	-	А-	256
<193>	Б	-Б	257
<256>	А-	БА	258
<258>	БА	А-Б	259
<045>	-	БА-	260
<195>	Г	-Г	261
<256>	А-	ГА	262
<203>	Л	А-Л	263
<256>	А-	ЛА	264
<204>	М	А-М	265
<256>	А-	МА	266
<262>	ГА	А-Г	267